

---

# WHAT REPOSITORIES REVEAL ABOUT AGENT-ASSISTED WORK: 3,649 ORGANISER-CREATED REPOSITORIES FROM A HACKATHON WITH A REQUIRED CODING AGENT

---

PREPRINT

**Talgar Bayan**  
[pending: affiliation]  
talgar.bayan@gmail.com

27 September 2026

## ABSTRACT

Research on AI coding agents often measures their use through traces left in software repositories, but such studies rarely know how their repositories were selected or which tool developers were meant to use. We study HackAlem AI, a five-hour hackathon held in Astana on 23 September 2026, at which every team had to use OpenAI Codex and the organisers created a public GitHub repository for registered teams in advance. We mirrored all 3,649 repositories with full history and reproduced the core counts with independent code. Organiser actions shaped the population: 446 repositories were archived before the event, and a batch of 1,040 created the evening before stayed almost unused. Event-time work reached 29.0% of all repositories and 48.2% of the rest. The median active team made 30.8% of its commits in the last hour. Codex left a detectable trace in 46.4% of active repositories and Claude Code, not required, in 24.9%. Test files appeared in 87.0%, and strings in the format of OpenAI keys reached the public history of 61 repositories, mostly through environment files. Rates measured from repositories depend on how an event is run and on each tool’s defaults. We release the derived data and the pipeline.

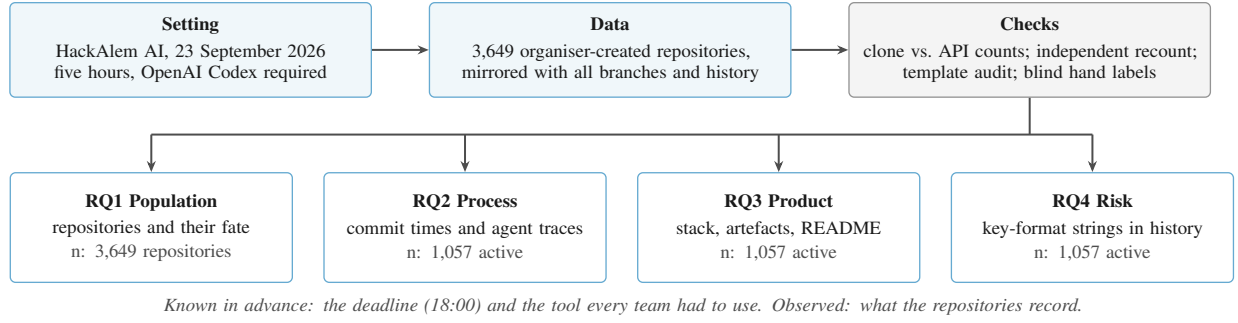
**Keywords** hackathons · AI coding agents · mining software repositories · human–AI collaboration · credential leakage · OpenAI Codex

## 1 Introduction

Coding agents such as OpenAI Codex and Claude Code edit whole repositories from natural-language instructions, and researchers increasingly study them through the traces they leave in version control: co-author trailers, branch names, pull requests and context files such as `AGENTS.md` [1–4]. These traces are partial. An agent that does not sign its commits leaves no signature, and a study that samples repositories from listings or from detected agent activity cannot see the repositories that were never used [3, 5, 6]. Two quantities are therefore usually unknown: the denominator, meaning every repository that could have shown activity, and the tool that developers were meant to use.

Hackathons offer a setting in which both can be known. They are time-bounded events at which small teams build prototypes [7], and their repositories have been studied at scale: how projects continue after an event, when their commits happen, and how much of their code is created during the event [8–12]. Studies of generative AI at hackathons, in turn, rely on interviews, surveys or single cases [13–16].

HackAlem AI, held in Astana on 23 September 2026, fixed both quantities. Teams had five hours to build a prototype and had to use Codex [17], and before the event the organisers created a public GitHub repository for registered teams in the BAITC–Hacks organisation. The organisation therefore holds every repository the organisers provisioned, including those that never received a commit, and every team worked under the same deadline and the same tool rule. We use this setting to ask what repository traces show about agent-assisted work, and what they miss, when the denominator and the intended tool are known. Our argument is that rates measured from repositories depend on how the event was run



**Figure 1:** Study design. The organisers’ repositories were mirrored and checked against the GitHub API, an independent recount and a template audit. Each research question has its own unit of analysis and denominator. The deadline and the required tool were fixed before the event, so the repositories can be compared with them.

and on each tool’s defaults, and that both have to be modelled before such rates are read as evidence about how people work with coding agents.

We mirrored all 3,649 repositories with their full history (Figure 1) and ask four questions:

**RQ1 Population.** Of the repositories the organisers created, how many carried work during the event, and how much of the gap reflects the organisers’ own actions?

**RQ2 Process.** How did teams spread their work over the five hours, and which traces do the required and the non-required agents leave?

**RQ3 Product.** What did teams build and document, and how does this vary between tracks?

**RQ4 Risk.** How did credentials reach the public history, and which repository set-ups go with them?

The paper makes four contributions:

- participation rates on a known denominator, separating the organisers’ actions (archiving before the event, a batch of new repositories) from the teams’ activity (RQ1, Section 5.1);
- a test of trace-based agent detection against a known required tool, showing how often context files, signatures and branch names reveal the required agent and how often they reveal another one (RQ2, Section 5.2);
- population-level prevalence, with intervals, of engineering artefacts, documentation languages and exposed credentials in agent-assisted prototypes, and the routes by which keys became public (RQ3 and RQ4, Sections 5.3 and 5.4);
- a derived dataset and a pipeline in which every reported number is generated from the data and the core counts are reproduced by independent code, with a checklist for studies of organiser-created repositories (Sections 4, 6 and 8).

## 2 Related work

**Which repositories a study sees.** Repository studies of hackathons have related project continuation to preparation, winning and team skills [8], found that most commits of hackathon projects fall in the first week and that few projects remain active months later [9], and measured how much code is written during an event and reused afterwards, starting from 22,183 projects listed on Devpost [10, 11]. HackRep collects 100,356 hackathon repositories and shows that event durations can be estimated from commit timestamps [12]. Samples of listed projects cannot contain registrations that produced no code. General cautions for mining GitHub apply as well: many repositories are inactive or personal [5], git histories can be rewritten and their timestamps are set by clients [18], mining tools disagree on basic counts [19], and representative samples are rare in software engineering research [6].

**Measuring agent use from repositories.** Before agents, ChatGPT use in open-source projects was found by matching messages that mention it and checking each match by hand [20]. Agents leave more explicit traces. The AIDev dataset collects pull requests authored by five agents [1]. Studies of projects that adopted agents report velocity gains, largest when the agent is the first AI tool a project uses, together with lasting increases in code complexity and static-analysis warnings [21, 22]. Robbes et al. estimate that 22.20–28.66% of 128,018 GitHub projects had adopted a coding agent by

**Table 1:** Facts about the event. Organiser and news sources were read on 24 September 2026 or later; the event window is our inference.

| Item            | Value                                                                                                                                    | Source       |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| Date and place  | 23 September 2026, Astana, in person                                                                                                     | [17]         |
| Duration        | five hours                                                                                                                               | [50–52]      |
| Event window    | 13:00–18:00 local time (UTC+5)                                                                                                           | this study   |
| Teams           | up to three members; individual registration allowed                                                                                     | [17]         |
| Tooling rule    | teams must use Codex during development                                                                                                  | [17]         |
| Capacity        | 2,500 seats; 6,586 applications                                                                                                          | [17, 49, 50] |
| Participants    | more than 2,500 from 21 countries and 129 universities                                                                                   | [51]         |
| Record attempt  | Guinness World Records, most participants in an agentic AI hackathon; previous record 2,089; result pending                              | [51]         |
| After the event | jury review of submitted projects 24–28 September; finalists present and winners are announced on 29 September; awards on 1 October 2026 | [17]         |
| Repositories    | public repositories in BAITC-Hacks, one per registered team                                                                              | this study   |

February 2026, and they describe the heuristics and pitfalls of detecting agents from traces [2, 3]. Agent context files have been characterised in open-source projects [4], and in a benchmark they lowered task success on average [23]. Agents are also evaluated on repository tasks [24, 25], and in 41% of real agent sessions collected from open-source developers the agent wrote virtually all committed code [26]. None of these studies knows which tool the developers were supposed to use.

**Time use under deadlines.** Commit timestamps show when developers work [27]. Students who start earlier tend to finish earlier and score higher [28, 29], most studies of time pressure report higher productivity and lower quality [30], and developers describe procrastination and its causes in interviews [31]. A hackathon compresses these questions into hours.

**Generative AI in hackathons and education.** In a controlled experiment, developers with GitHub Copilot finished a task faster than a control group [32]. In a field study, perceived productivity was linked to how often suggestions were accepted [33]. Observational work describes separate acceleration and exploration modes [34], and developers surveyed report both benefits and trouble controlling the tools [35]. At hackathons, participants use generative AI for coding, learning and documentation and check its output under time pressure [13]. Novices can deliver working demonstrations while engaging little with engineering practice [14], a rule against manual editing shapes prompting and debugging [15], and a university case discusses educational opportunities and risks [16]. Building software mainly by prompting, often called vibe coding, has been described as a trade between speed and technical debt [36], and computing educators have reviewed what these tools mean for teaching [37, 38].

**Secrets in public repositories.** Secrets leak to public GitHub repositories often and remain there [39]. Detection methods have been proposed [40], a labelled dataset supports the evaluation of detectors [41], and a comparison of detectors found Gitleaks to have the highest recall (88%) with lower precision (46%) [42]. Developers who leaked secrets describe remediation as hard [43]. Code completion models can reproduce credentials seen in training [44], and participants with an AI assistant wrote less secure code while believing it more secure [45]. For AI-built software, LLM API keys leak from mobile apps [46], and agent-generated and vibe-coded applications contain vulnerabilities [47, 48]. We found no repository-level study of a hackathon in which repositories were provisioned for registered teams and the tool was fixed in advance.

### 3 The event and its repositories

Table 1 lists the facts about the event that we use, with their sources. Registration was free and people could register alone [17]. News reports describe long queues at the entrance and registered participants who could not get in; the organisers said that participants had been told in advance that seats were limited [49, 50]. The event had 12 tracks, each set by an industry or public-sector partner (Table 3).

Each repository was named `hack-<id>-<team>` and began with an organiser commit titled “Initial commit” that added a README of two lines. The organisers archived the repositories, which made them read-only: 3,201 were archived between 18:01 and 18:51 on the event day, and all 3,649 were archived when we collected the data. Section 5.1 shows that the rest were archived before the event began.

## 4 Data and methods

**Collection.** On 24 September 2026 we listed all repositories of the organisation through the GitHub REST API, including their creation and last-update times, collected default-branch commit counts through GraphQL, and made mirror clones of all 3,649 repositories. The clones keep every branch, every pull-request reference and the full history. All clones completed, and because the repositories were archived their content could no longer change.

**Units and definitions.** A *team commit* is a commit reachable from any branch, counted once per SHA, other than the organisers’ template commit. We find the template commit with a rule that does not depend on the author account: a commit without parents, titled exactly “Initial commit”, made within 120 s of the repository’s creation, that adds two lines to one file and deletes none. The *event window* is 13:00–18:00 local time on 23 September 2026, measured on committer timestamps. It is inferred from commit and archive times together with press reports of a five-hour event, since we found no official schedule. An *active repository* has at least one team commit in the window. We read a last-update time before the start of the window as *archived before the event*. Archiving updates this time, and none of these repositories received a commit during the event. The *evening batch* consists of the repositories created between 18:00–20:00 on 22 September. We treat it as an organiser action, because up to 15 repositories were created per minute then, against at most 8 in any minute of earlier days. A repository stands for one registration of up to three people, and its creation date stands in for the registration date; we make no claims about individuals. RQ1 uses all 3,649 repositories. RQ2–RQ4 use the 1,057 active repositories, README measures use the 1,033 of them with a written README, and commit measures use their team commits in the window unless stated.

**Measures.** From the final state of each default branch we record the primary language by file size, frameworks from package manifests, and LLM use from manifests, import statements and API endpoints, excluding vendored folders. Engineering artefacts are test files (by file and folder name), a Dockerfile, a Compose file, a CI workflow and a deployment configuration. We record three kinds of agent trace for each tool:

- *Context files:* AGENTS.md at any depth (the file Codex reads, which other agents read too); CLAUDE.md or a .claude folder; Cursor rules; Copilot instructions; GEMINI.md.
- *Commit signatures:* a co-author trailer or a “generated with” line that names an agent, or an author or committer that is an agent account or carries exactly a tool’s name.
- *Branch names* that start with an agent’s name.

Pull-request references are a workflow trace, reported separately. Generic bot accounts, among them the organisers’ application, are not counted as agents.

For the README at the repository root we remove code, links and markup and classify the prose by its letters:

- *Kazakh* if at least 50% of the letters are Cyrillic and at least 3% of those are specific to Kazakh (nine letters that Russian does not use);
- *Russian* if at least 50% of the letters are Cyrillic otherwise;
- *English* if at least 80% are Latin;
- *mixed* in all other cases.

The script of commit subject lines is classified in the same way. README content has been categorised in earlier work [53]; we record README length and whether the README has run or setup instructions (a heading on installation, setup or usage, or a command inside a code block), images, a link to a deployed application, and a mention of a coding agent.

For credentials we ran Gitleaks [54] version 8.30.1 over every reference of the 1,254 clones with team commits, with redacted output. We kept privately the rule, file, commit, line, date and whether the matched string was still present at the final state. We never stored or printed a matched value and never tested one against a service. For provider-specific rules, a string counts as a placeholder if it contains placeholder text (such as a run of x, “your”, “example” or angle brackets) or has a character entropy below 3.5 bits.

**Track labels.** Track labels come from an earlier per-repository table that the author produced for a public report on the event. We checked them against README text with keyword rules. Where the keyword rules gave one clear track, they agreed with the label for 933 of 946 active repositories (98.6%,  $\kappa = 0.98$ ). We read every disagreement and corrected 7 labels. Of the labelled repositories, 19 had no README text to check, and a further 73 active repositories stayed unassigned.

**Table 2:** Validation and sensitivity checks.

| Check                                                                      | Result                                       |
|----------------------------------------------------------------------------|----------------------------------------------|
| Default-branch commit counts, clones vs. GitHub API                        | 3,649 of 3,649 equal                         |
| Independent recount of team commits / window commits / hours with commits  | 3,649 / 3,649 / 3,649 of 3,649 equal         |
| Template commit found by the account-independent rule                      | 3,615 repositories; 34 rewrote their history |
| Template commits by a second organiser account                             | 16 of 402 audited repositories               |
| Consistency with the earlier table (team commits / window commits / hours) | 3,641 / 3,644 / 3,648 of 3,649 equal         |
| Active repositories with author time instead of committer time             | 1,057 (committer time: 1,057)                |
| Active repositories with the window shifted by $-60 / -30 / +30 / +60$ min | 1,026 / 1,040 / 1,060 / 1,059                |
| Track labels vs. keyword rules (active, one clear track)                   | 933 of 946 agree; $\kappa = 0.98$            |
| README language rule vs. blind hand labels (81 READMEs)                    | [pending: labelling in progress]             |

**Validation.** Table 2 lists the checks. Default-branch commit counts from the clones equal those from the GitHub API for every repository, and an independent script that shares no code with the pipeline reproduced team commits, window commits and hours with commits for every repository. While spot-checking repositories on GitHub we found template commits made by a second organiser account. An audit of all 402 repositories this could affect found 16 such repositories, and the account-independent rule now handles them. Commits reachable only through pull-request references (427 commits in 117 repositories) are not team commits. The active count hardly moves when the window is shifted or author time replaces committer time.

**Statistics.** The repositories are the complete set for this event, so intervals and tests describe how precisely the observed shares reflect the process that produced them, not sampling from other events [6]. Proportions carry Wilson 95% confidence intervals, and distributions are summarised by the median and interquartile range (IQR). We compare groups with  $\chi^2$  tests (effect size Cramér’s  $V$ , reporting cells with small expected counts) and Mann–Whitney tests (Cliff’s  $\delta$ ), adjust one association with logistic regression and one with a Mantel–Haenszel odds ratio, and compare tracks with a Kruskal–Wallis test ( $\varepsilon^2$ ). The three tests about AGENTS.md form one family with Holm correction; the other tests stand alone. We fixed the tests after exploring descriptive numbers and before running them; the plan, the deviations from it and the reasons are released with the data, and analyses added later are labelled *exploratory*. Every number in the paper is generated from the data by the released scripts.

**Use of AI tools.** The collection and analysis code, the figures and a first draft of this text were produced with an AI coding agent (Claude Code, Anthropic) under the author’s direction. The author set the research questions, definitions and analysis plan and checked the outputs. Two internal reviews, also run with the agent, found errors that are corrected in this version and listed in the released deviations file. [pending: author to name any other AI tools used]

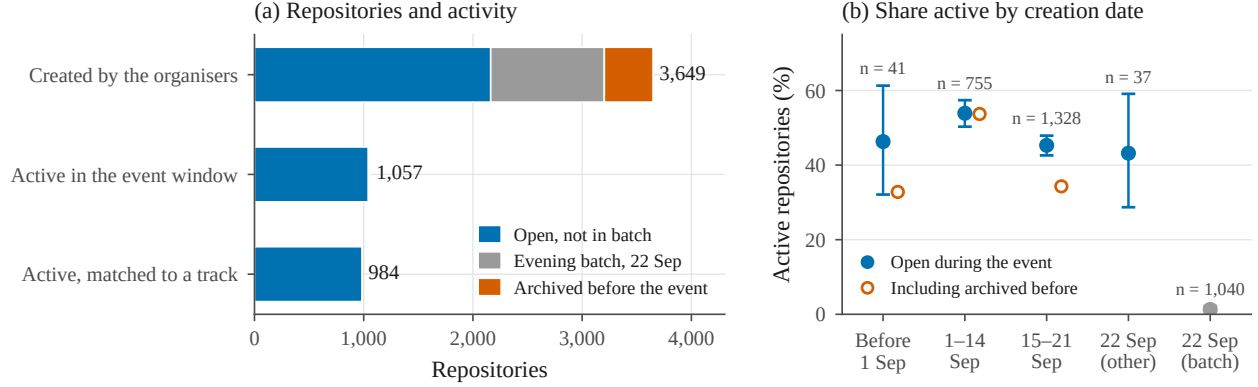
## 5 Results

### 5.1 RQ1: Population

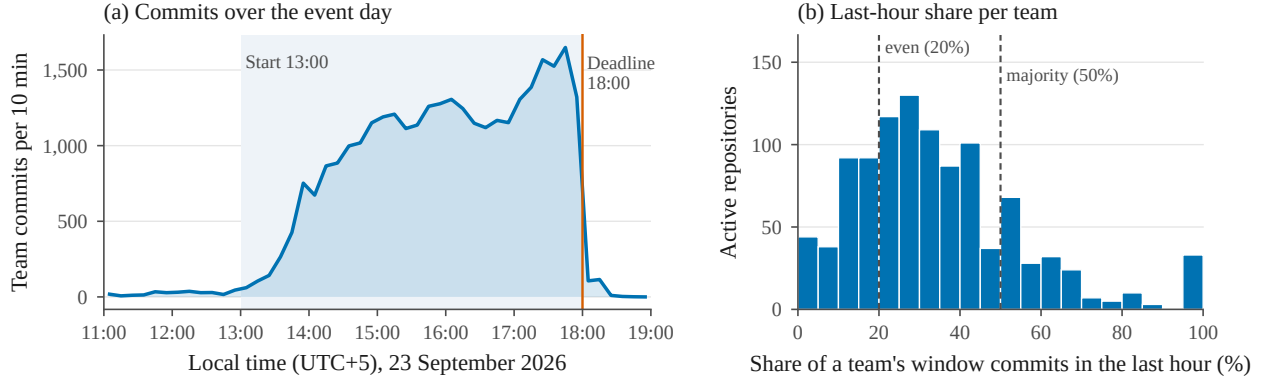
The organisers created 3,649 repositories, but not all of them were open during the event (Figure 2a). Of these, 446 (12.2%) had been archived before it began, in three bulk operations: 126 repositories between 18:30 and 18:32 on 15 September, 56 on 17 September, and 242 between 20:01 and 20:05 on 22 September, with 22 more on other days. Most of them (426) had been created between 15 September and 21 September. The last operation came minutes after the evening batch of 1,040 new repositories. Of the batch, 14 (1.3%) became active. None of the repositories archived before the event became active, although 157 of them already held team commits from before it.

Over all repositories, 1,254 (34.4%) have team commits and 1,057 (29.0%, 95% CI 27.5–30.5) are active. Among the 3,203 repositories open during the event, the active share is 33.0%. Among those that were open and not in the batch, it is 48.2% (1,043 of 2,163; 95% CI 46.1–50.3). Among open repositories outside the batch, activity varied little with creation date (Figure 2b), from 43.2% for those created on 22 September to 53.9% for those created between 1 September and 14 September ( $\chi^2(3) = 14.9$ ,  $p = 0.002$ ,  $V = 0.08$ ). The lower share among all repositories created between 15 September and 21 September (34.3% against 45.3% of the open ones) comes from the archiving.

Of the 197 repositories with team commits but none in the window, 194 had all of them before the event and 157 were archived before it. The active repositories hold at most 2,740 distinct author identities (median 3 per repository). This is an upper bound on the number of people, because one person can commit under several e-mail addresses; press reports give more than 2,500 participants [51]. Of the active repositories, 984 (93.1%) could be assigned to a track.



**Figure 2: Population.** (a) Repositories created by the organisers, split into those open during the event and not in the evening batch, the batch, and those archived before the event; below them, the active repositories and those matched to a track. (b) Share of repositories that became active by creation date, with 95% Wilson intervals, for repositories open during the event (filled) and for all repositories, including those archived before the event (open circles).



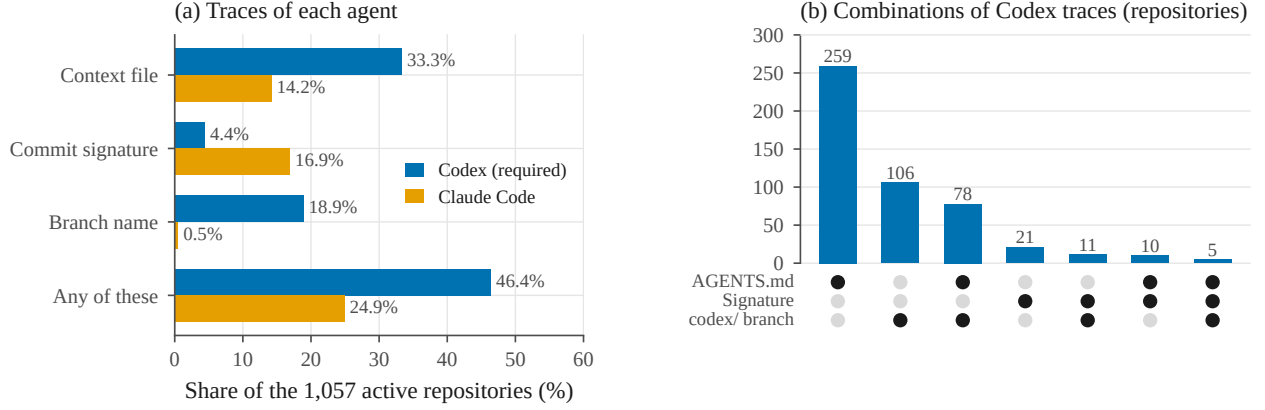
**Figure 3: Process.** (a) Team commits per ten minutes on the event day, all branches of all repositories; the shaded band is the event window. (b) Distribution over the active repositories of the share of each team's window commits made in the last hour. Dashed lines mark an even spread and a majority.

**Answer to RQ1.** Organiser actions account for most of the gap between repositories and activity. The share of repositories with event-time work rises from 29.0% of all repositories to 48.2% once the repositories archived before the event and the evening batch are set aside, and among the rest the creation date matters little.

## 5.2 RQ2: Process

**Timing.** Active repositories received 30,418 team commits in the window. Activity rose during the first hour and stayed high through the afternoon, and the last hour held 28.8% of all window commits. The busiest ten minutes were 17:40–17:50, with 1,649 commits (Figure 3a). Per team, the median share of commits made in the last hour was 30.8% (IQR 19.4–43.8%), against 20% if work had been spread evenly (Figure 3b). In that hour, 163 teams (15.4%, 95% CI 13.4–17.7) made most of their commits. At the same time, 79.3% of teams committed in at least four of the five hours and 44.5% in all five. The median team made its first window commit 61 minutes after the start (IQR 43–94), its last 9 minutes before the deadline (IQR 3–20), and 19 commits in all (IQR 10–35). Some active repositories also had commits from before the event (173, of which 33 changed more than a thousand lines) or after the deadline (73). Clustering the teams' hourly profiles, an exploratory analysis, separated a late group of 323 teams, but with weak structure: the silhouette was 0.30, against 0.19 for data with no team structure [55]. We therefore describe the distribution and do not assign teams to types.

**Traces of agents.** Codex, the required tool, left at least one trace in 490 active repositories (46.4%, 95% CI 43.4–49.4; Figure 4a). The traces were an AGENTS.md file (33.3%), a branch starting with codex (18.9%), and rarely a commit signature (4.4%), and they seldom occurred together (Figure 4b). Claude Code, which was not required, left a trace in 24.9% of active repositories, mostly through signed commits (16.9%) and CLAUDE.md files (14.2%). Claude Code



**Figure 4:** Traces of coding agents in the active repositories. (a) Share with each kind of trace for Codex, which every team had to use, and for Claude Code. (b) Combinations of the three Codex traces; dots mark the traces present, and bars count repositories with exactly that combination.

without any trace of Codex appears in 82 repositories (7.8%). Other agents appear in few repositories (Cursor 20, Copilot 5, Gemini 2). In 44.9% of active repositories we found no trace of any agent. Of all team commits, 3,953 (12.4%) carry an agent signature: 3,503 name Claude and 332 name Codex. Pull-request references, a workflow trace, occur in 35.9% of active repositories, 143 of them with a Codex branch.

In an exploratory comparison, repositories with an `AGENTS.md` file had more window commits (median 28 vs. 16;  $\delta = 0.36$ , Holm-adjusted  $p < 0.001$ ) and more hours with commits (median 5 vs. 4;  $\delta = 0.26$ ,  $p < 0.001$ ), and they more often contained test files (95.7% vs. 82.7%,  $p < 0.001$ ). The difference in tests remains after adjusting for the number of window commits (odds ratio 3.51, 95% CI 1.99–6.17). Teams that work with agents may produce both the file and the tests, and teams that prepared more may differ in other ways, so these are associations only.

**Answer to RQ2.** Most teams worked throughout the afternoon, with a moderate rise towards the deadline; a minority made most of their commits in the last hour. Under a rule that required Codex, the usual trace heuristics found Codex in fewer than half of the active repositories, and they found a tool that was not required in a quarter of them.

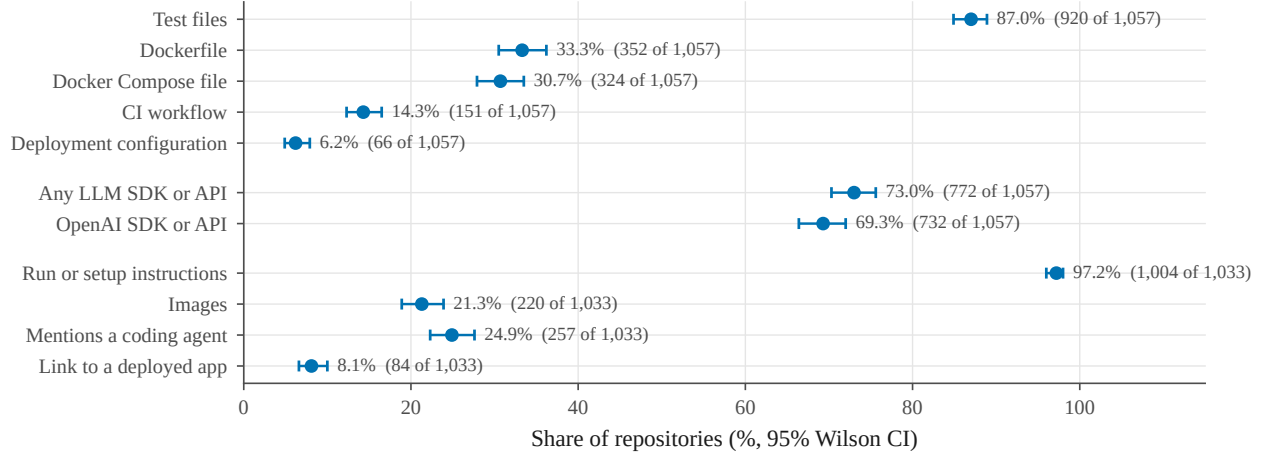
### 5.3 RQ3: Product

Python was the primary language of 58.5% of active repositories, followed by TypeScript (21.7%) and JavaScript (12.5%). The most common frameworks were React (46.5%), FastAPI (43.2%), Vite (32.8%), Tailwind (21.2%) and Next.js (16.5%). An LLM SDK or API call appeared in 73.0% of repositories, OpenAI’s in 69.3%, and each other provider in at most 2.9%. Test files were present in 87.0% of repositories, a Dockerfile in 33.3%, a CI workflow in 14.3% and a deployment configuration in 6.2% (Figure 5). The shares hardly change without the repositories that had commits before the event or after the deadline (test files 86.6% of 831). We did not run the tests.

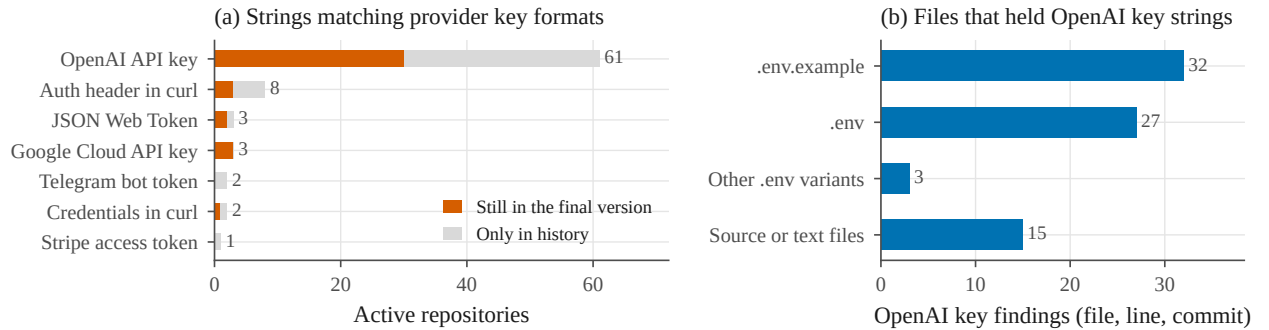
Of the 1,033 active repositories with a written README, 88.1% were in Russian, 9.3% in English, 2.1% in Kazakh and 0.5% mixed; Kazakh-specific letters appeared somewhere in 14.2%. Commit subjects, by contrast, were mostly in Latin script (87.5% of non-merge window commits; Cyrillic 12.1%). READMEs were long, with a median of 1,645 words (IQR 1,049–2,413) and 16 headings. Of these READMEs, 97.2% gave run or setup instructions, 21.3% included images, 24.9% mentioned a coding agent and 8.1% linked to a deployed application.

Table 3 describes the tracks. The primary language differed between tracks ( $\chi^2(33) = 201.1$ ,  $p < 0.001$ ,  $V = 0.26$ ; 8 of 48 cells have expected counts below five). Python ranged from 30.4% of repositories in the city-simulator track to 95.7% in the telecom track. Window commits per team varied little (medians 16–31;  $H(11) = 19.0$ ,  $p = 0.062$ ,  $\epsilon^2 = 0.01$ ). Kazakh READMEs were rare in every track, including the one whose task was to take minutes from Kazakh, Russian or mixed meeting audio (4.6%).

**Answer to RQ3.** In five hours most teams produced a Python or TypeScript application that called an LLM, together with test files and a long README, mostly in Russian. Tracks differed in language choice but not clearly in the amount of work.



**Figure 5:** Product. Engineering artefacts and LLM use in the 1,057 active repositories, and README features in the 1,033 of them with a written README; shares with 95% Wilson intervals.



**Figure 6:** Risk. (a) Active repositories with strings that match provider-specific key formats, split by whether the string is still present in the final version. (b) Files that held the OpenAI key strings; a finding is one file, line and commit, so a key can be counted more than once.

## 5.4 RQ4: Risk

Gitleaks’ generic rule produced 4,597 findings in 131 repositories, 93.5% of them in data files (.json, .jsonl, .txt or .csv). We treat these as low-precision and do not analyse them further. Provider-specific rules produced 127 findings, of which 10 were placeholders. The other 117 lie in 73 active repositories (6.9%), and in 36 (3.4%) the string is still present in the final version (Figure 6a). Most are OpenAI API keys: 77 findings in 61 repositories (5.8%, 95% CI 4.5–7.3), 30 of them with the string still present. By the commit dates Gitleaks reports, all were committed during the event window.

The OpenAI key strings sat mainly in environment files (Figure 6b): 32 findings in .env.example and 27 in .env. A .env.example file is a template meant to be committed, so a real key placed there becomes public even when .gitignore excludes .env. .gitignore covered .env in 83.6% of active repositories, and a .env.example file was present in 74.3%. Only 3.2% committed a .env file. In an exploratory analysis, key strings were not clearly more frequent where .gitignore covered .env (6.1% vs. 4.0%,  $p = 0.287$ ). Adjusting for LLM use, which went with both (coverage of .env in 90.9% of repositories with an LLM SDK against 63.9% without), gave a Mantel–Haenszel odds ratio of 1.03 (95% CI 0.45–2.37). Keys were not committed more often by agents: 5 of the 73 commits that added an OpenAI key string (6.8%) carry an agent signature, against 9.0% of all window commits in the same repositories. We did not test whether any key worked.

**Answer to RQ4.** Strings in the format of OpenAI keys reached the public history of 5.8% of active repositories, all during the event, mainly through committed template and environment files. They were not tied to agent-signed commits or to missing .gitignore rules.

**Table 3:** The 12 tracks with the active repositories matched to each, their median number of window commits, and the shares with test files, with Python as primary language and with a Kazakh README (of written READMEs). Partners are named in team READMEs; task descriptions are our summaries of those READMEs.

|     | Track               | Partner            | Task                                                                 | Repos | Commits (median) | Tests (%) | Python (%) | Kazakh (%) |
|-----|---------------------|--------------------|----------------------------------------------------------------------|-------|------------------|-----------|------------|------------|
| T1  | Energy              | Samruk-Kazyna      | Agent that forecasts wind-farm output from weather data              | 74    | 25               | 90.5      | 83.8       | 2.7        |
| T2  | Finance             | Freedom            | Rebuild an organised group from a transaction network                | 115   | 21               | 87.0      | 74.8       | 1.8        |
| T3  | Management          | Halyk Bank         | Guide linking employee skills to growth steps                        | 89    | 21               | 93.3      | 60.7       | 1.1        |
| T4  | Telecom             | Beeline            | Agent choosing tariff offers for subscribers                         | 46    | 18               | 76.1      | 95.7       | 0.0        |
| T5  | Logistics           | ekt.kz             | Supplier orders to replenish stock, approved by a manager            | 80    | 22               | 91.2      | 67.5       | 1.2        |
| T6  | Creative industries | Firebird           | Choose event contractors and explain the choice                      | 113   | 16               | 88.5      | 63.7       | 1.8        |
| T7  | Education           | AI Sana            | Turn a business problem into a task for student teams                | 145   | 20               | 86.2      | 31.0       | 1.4        |
| T8  | Innovation          | Samruk-Kazyna      | Minutes and action items from Kazakh, Russian or mixed meeting audio | 65    | 17               | 89.2      | 87.7       | 4.6        |
| T9  | Communications      | Halyk Bank         | Voice agent that routes calls in an insurance contact centre         | 29    | 31               | 79.3      | 72.4       | 0.0        |
| T10 | Trade               | ekt.kz             | Shopping assistant: products, alternatives, prices, stock            | 71    | 19               | 90.1      | 49.3       | 5.6        |
| T11 | Special             | Kazakhtelecom      | Compare organisation charts before and after a reorganisation        | 42    | 20               | 92.9      | 61.9       | 2.4        |
| T12 | Special             | Astana Innovations | City simulator that splits a budget across districts                 | 115   | 23               | 94.8      | 30.4       | 0.9        |

**Table 4:** Answers to the research questions, with the unit and denominator of each.

| RQ  | Unit (n)                                             | Answer and key estimates                                                                                                                                                                                         |
|-----|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RQ1 | repositories (3,649)                                 | Organiser actions explain most of the gap: 446 archived before the event and a batch of 1,040 with 1.3% active. Active: 29.0% of all, 33.0% of open, 48.2% of open outside the batch; creation date $V = 0.08$ . |
| RQ2 | active repositories (1,057); window commits (30,418) | Work throughout the afternoon: median last-hour share 30.8%, majority in the last hour for 15.4%. Codex traces in 46.4%; Claude Code traces in 24.9%; no trace in 44.9%.                                         |
| RQ3 | active repositories (1,057); written READMEs (1,033) | Python 58.5%; LLM use 73.0%; test files 87.0%; Dockerfile 33.3%; READMEs in Russian 88.1%, Kazakh 2.1%; median 1,645 words.                                                                                      |
| RQ4 | active repositories (1,057); key findings (77)       | OpenAI key strings in 5.8% (2.8% still present), mostly in <code>.env.example</code> and <code>.env</code> ; agent-signed 6.8% vs. 9.0% baseline; no association with <code>.gitignore</code> (odds ratio 1.03). |

## 6 Discussion

Table 4 summarises the answers. We discuss them in three themes and then turn to implications.

### 6.1 Denominators decide rates

The same event yields an activity rate of 29.0%, 33.0% or 48.2%, depending on whether repositories closed by the organisers and a batch created the evening before are counted. Taken at face value, all repositories suggest that the

date of registration mattered a great deal ( $\chi^2(3) = 608.2$ ,  $V = 0.41$ ); among open repositories outside the batch the effect is small ( $V = 0.08$ ). We do not know why the organisers archived repositories in bulk or created the batch; possible reasons include withdrawn or duplicate registrations and the allocation of limited seats, which the news reports describe [49, 50]. The lesson for researchers is general: organiser-created repositories record the organisers’ actions as well as the teams’, and archive times, creation bursts and template commits have to be reconstructed before any rate is read as participation. Listing-based datasets face the opposite problem, since they never see the empty repositories at all [10, 11].

## 6.2 What traces reveal under a known mandate

Every team was told to use Codex, yet the trace heuristics used in repository studies detected Codex in 46.4% of active repositories. If all teams complied, this is an upper bound on what these heuristics recover for a tool whose commits here rarely carried a signature. The `AGENTS.md` file and the `codex` branch carried most of the signal, and signatures almost none. Claude Code adds a co-author trailer to its commits by default [56], and its signatures outnumbered those of Codex by far. Adoption studies that rely on signatures [1, 2] will therefore count tools unevenly. This is the partial observability that Robbes et al. describe [3]; here it can be sized against a known intended tool. The Claude Code traces also give a lower bound on the use of a tool that the rules did not require, in 24.9% of active repositories. The rule named Codex; whether it excluded other tools is not stated in the sources we read, so we do not read these traces as breaches.

The timing is hard to compare with course assignments, where students who start late tend to do worse [28, 29], because the time frame is so different. Most teams here were active in most hours and the rise towards the deadline was moderate. Without a comparable event held without agents, we cannot say whether agents changed this pattern.

## 6.3 Cheap artefacts, costly mistakes

Test files, long READMEs with run instructions and, in a third of repositories, Docker files were produced within five hours. One explanation, which our data cannot test, is that agents make such artefacts cheap; in many real agent sessions the agent writes virtually all committed code [26]. Their presence says nothing about whether the tests run or pass, and quality risks of agent-written code are documented elsewhere [22, 47, 48]. The documentation was written mostly in Russian, and Kazakh was rare even in the track built around Kazakh audio. Language models support Kazakh less well than high-resource languages [57], which may push teams that work through agents towards Russian or English; the audience that teams wrote for may matter as much, and our data cannot separate the two.

The keys that became public followed a route that simple measures would have closed. They sat in templates that are meant to be committed, they were not tied to agent-signed commits, and `.gitignore` rules did not prevent them. Repository templates that ship a `.gitignore` and a placeholder-only `.env.example`, organisation-level secret scanning with push protection, and short-lived keys issued for the event are consistent with earlier recommendations [39, 42, 43].

## 6.4 Implications

For organisers, creating repositories after check-in would turn the organisation into a record of participants rather than of registrations, and the measures above would reduce exposure of the keys they issue. For researchers, we suggest a short checklist for organiser-created repositories: reconstruct archive and creation events, audit template commits across organiser accounts, mine every branch and pull-request reference, state each denominator, and combine several kinds of trace for each tool, reporting what each one recovers. For educators who run agent-assisted events, the traces show which tools students chose, not how well they used them.

# 7 Threats to validity

We follow the empirical standards for repository mining [58] in organising the threats.

**Construct validity.** A repository stands for a registration, and author identities overstate the number of people, so we make no claims about individuals. Commit counts measure activity, not effort or quality, and test files show an intention to test, not working tests. Agent traces are partial by design; we report what each one recovers and treat none as a measure of use. The README language rule is a letter heuristic and misses Kazakh typed with Russian letters. Its accuracy against blind hand labels is [pending: being measured]. Credential findings match key formats and were not tested. The placeholder filter was not validated against hand labels. Gitleaks’ rules fix key formats, so keys in other formats are missed [42].

**Internal validity.** The event window is inferred, not taken from a schedule. It is supported by the archive times, and moving it changes the active count little (Table 2). Features are measured at the final state, which includes pre-event and post-deadline commits in some repositories; excluding those repositories barely changes the shares. Reading bulk archiving from last-update times is an inference. It is consistent with the absence of any commit in those repositories during the event. The associations with `AGENTS.md` and with `.gitignore` are observational. Track labels were produced and checked by one person.

**External validity.** We study one in-person event in one country, with one required agent and a five-hour window. Other events differ in duration, rules, tools and participants, and the share of empty repositories depends on how an organiser provisions them.

**Reliability.** The collection, analysis and figure code, the analysis plan with its deviations, and the derived data are released. The repositories are archived and public, and we list the commit each clone ended at, so the data can be collected again.

## 8 Ethics and data availability

All data come from public repositories and public web pages, and we accessed no private system. Author names and e-mail addresses were matched against agent patterns in memory and never stored. Distinct authors were counted through a salted hash whose salt was never saved. The released data use random identifiers; because the source repositories are public, this is pseudonymisation and not anonymisation. Secret values were redacted by the scanner, never stored or printed by our code and never tested, and we report credentials only in aggregate. We informed the organisers privately about the repositories containing credentials before publication [pending: confirm after contacting the organisers]. We are not affiliated with the organisers, and this paper reports no official results of the event. [pending: ethics review or exemption, once the affiliation is set]

The derived dataset, the pipeline, the analysis plan and the deviations from it are available at <https://github.com/tbayan/HackaleMAI-Repo-Data-Mining> [pending: archived release with DOI]. The analysis of the jury’s decisions was pre-registered before any finalist or winner was announced [59] [pending: OSF registration link].

## 9 Conclusion

A hackathon at which the organisers created the repositories and fixed the tool lets repository measures be checked against what is known in advance. At HackAlem AI, the organisers’ own actions moved the activity rate by 19.2 percentage points. The required agent was detected in fewer than half of the active repositories and a non-required one in a quarter. Keys became public mainly through committed template and environment files, and agent-signed commits and missing ignore rules played no clear part. Studies of agent-assisted work that start from repositories should report how their population was formed and what each trace can recover. After the awards, we will test the pre-registered hypotheses about the jury’s decisions and compare these repositories with hackathon repositories from before coding agents [12].

## References

- [1] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. arXiv preprint arXiv:2507.15003, 2025. URL <https://arxiv.org/abs/2507.15003>.
- [2] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. Agentic Much? Adoption of Coding Agents on GitHub. *ACM Transactions on Software Engineering and Methodology*, June 2026. doi:10.1145/3822180.
- [3] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. Promises, Perils, and (Timely) Heuristics for Mining Coding Agent Activity. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR ’26, page 496–507. ACM, April 2026. doi:10.1145/3793302.3793375.
- [4] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Lee-laprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. *ACM Transactions on Software Engineering and Methodology*, September 2026. doi:10.1145/3840295.

- [5] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, ICSE '14, page 92–101. ACM, May 2014. doi:[10.1145/2597073.2597074](https://doi.org/10.1145/2597073.2597074).
- [6] Sebastian Baltes and Paul Ralph. Sampling in software engineering research: a critical review and guidelines. *Empirical Software Engineering*, 27(4), April 2022. doi:[10.1007/s10664-021-10072-8](https://doi.org/10.1007/s10664-021-10072-8).
- [7] Connie W. Chau and Elizabeth M. Gerber. On Hackathons: A Multidisciplinary Literature Review. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, page 1–21. ACM, April 2023. doi:[10.1145/3544548.3581234](https://doi.org/10.1145/3544548.3581234).
- [8] Alexander Nolte, Irene-Angelica Chounta, and James D. Herbsleb. What Happens to All These Hackathon Projects? Identifying Factors to Promote Hackathon Project Continuation. *Proceedings of the ACM on Human-Computer Interaction*, 4(CSCW2):1–26, October 2020. doi:[10.1145/3415216](https://doi.org/10.1145/3415216).
- [9] Lukas McIntosh and Caroline D. Hardin. Do Hackathon Projects Change the World? An Empirical Analysis of GitHub Repositories. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*, SIGCSE '21, page 879–885. ACM, March 2021. doi:[10.1145/3408877.3432435](https://doi.org/10.1145/3408877.3432435).
- [10] Ahmed Imam, Tapajit Dey, Alexander Nolte, Audris Mockus, and James D. Herbsleb. The Secret Life of Hackathon Code Where does it come from and where does it go? In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, page 68–79. IEEE, May 2021. doi:[10.1109/msr52588.2021.00020](https://doi.org/10.1109/msr52588.2021.00020).
- [11] Ahmed Samir Imam Mahmoud, Tapajit Dey, Alexander Nolte, Audris Mockus, and James D. Herbsleb. One-off events? An empirical study of hackathon code creation and reuse. *Empirical Software Engineering*, 27(7), September 2022. doi:[10.1007/s10664-022-10201-x](https://doi.org/10.1007/s10664-022-10201-x).
- [12] Sjoerd Halmans, Lavinia Paganini, Alexander Serebrenik, and Alexander Nolte. HackRep: A Large-Scale Dataset of GitHub Hackathon Projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR '26, page 579–583. ACM, April 2026. doi:[10.1145/3793302.3793305](https://doi.org/10.1145/3793302.3793305).
- [13] Wangyiyao Zhou, Alexander Serebrenik, and Alexander Nolte. Quick Build, Careful Check? Generative AI Use in Hackathons. In *Proceedings of the 2026 International Conference on Game Jams Hackathons and Game Creation Events*, ICGJ 2026, page 23–27. ACM, August 2026. doi:[10.1145/3833089.3833091](https://doi.org/10.1145/3833089.3833091).
- [14] Kiev Gama, Filipe Calegario, Victoria Jackson, Alexander Nolte, Luiz Morais, and Vinicius Cardoso Garcia. “Can you feel the vibes?”: An exploration of novice programmer engagement with vibe coding. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*, ICSE-SEET '26, page 485–496. ACM, April 2026. doi:[10.1145/3786580.3786992](https://doi.org/10.1145/3786580.3786992).
- [15] Ashley J. Chen, Yijia Cao, Minghao Shao, Ramesh Karri, and Muhammad Shafique. Code for All: Educational Applications of the “Vibe Coding” Hackathon in Programming Education across All Skill Levels. arXiv preprint arXiv:2604.22747, 2026. URL <https://arxiv.org/abs/2604.22747>.
- [16] Ramteja Sajja, Carlos Erazo Ramirez, Zhouyayan Li, Bekir Z. Demiray, Yusuf Sermet, and Ibrahim Demir. Integrating Generative AI in Hackathons: Opportunities, Challenges, and Educational Implications. *Big Data and Cognitive Computing*, 8(12):188, December 2024. doi:[10.3390/bdcc8120188](https://doi.org/10.3390/bdcc8120188).
- [17] HackAlem AI. HackAlem AI — Astana, 23.09.2026, 2026. URL <https://hackalem.ai/>. Official event website. Accessed 24 September 2026.
- [18] Christian Bird, Peter C. Rigby, Earl T. Barr, David J. Hamilton, Daniel M. German, and Prem Devanbu. The promises and perils of mining git. In *2009 6th IEEE International Working Conference on Mining Software Repositories*, page 1–10. IEEE, May 2009. doi:[10.1109/msr.2009.5069475](https://doi.org/10.1109/msr.2009.5069475).
- [19] Nicole Hoess, Carlos Paradis, Rick Kazman, and Wolfgang Mauerer. Does the Tool Matter? Exploring Some Causes of Threats to Validity in Mining Software Repositories. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, page 645–656. IEEE, March 2025. doi:[10.1109/saner64311.2025.00067](https://doi.org/10.1109/saner64311.2025.00067).
- [20] Rosalia Tufano, Antonio Mastropaolo, Federica Pepe, Ozren Dabic, Massimiliano Di Penta, and Gabriele Bavota. Unveiling ChatGPT’s Usage in Open Source Projects: A Mining-based Study. In *Proceedings of the 21st International Conference on Mining Software Repositories*, MSR '24, page 571–583. ACM, April 2024. doi:[10.1145/3643991.3644918](https://doi.org/10.1145/3643991.3644918).
- [21] Shyam Agarwal, Hao He, and Bogdan Vasilescu. AI IDEs or Autonomous Agents? Measuring the Impact of Coding Agents on Software Development. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR '26, page 857–862. ACM, April 2026. doi:[10.1145/3793302.3793589](https://doi.org/10.1145/3793302.3793589).

- [22] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR '26, page 181–193. ACM, April 2026. doi:[10.1145/3793302.3793349](https://doi.org/10.1145/3793302.3793349).
- [23] Thibaud Gloaguen, Niels Mündler, Mark Müller, Veselin Raychev, and Martin Vechev. Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? arXiv preprint arXiv:2602.11988, 2026. URL <https://arxiv.org/abs/2602.11988>.
- [24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-world GitHub Issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL [https://proceedings.iclr.cc/paper\\_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html). Preprint: doi:[10.48550/arXiv.2310.06770](https://doi.org/10.48550/arXiv.2310.06770).
- [25] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37*, NeurIPS 2024, page 50528–50652. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi:[10.52202/079017-1601](https://doi.org/10.52202/079017-1601).
- [26] Joachim Baumann, Vishakh Padmakumar, Xiang Li, John Yang, Diyi Yang, and Sanmi Koyejo. SWE-chat: Coding Agent Interactions From Real Users in the Wild. arXiv preprint arXiv:2604.20779, 2026. URL <https://arxiv.org/abs/2604.20779>.
- [27] Maëlick Claes, Mika V. Mäntylä, Miikka Kuuttila, and Bram Adams. Do programmers work at night or during the weekend? In *Proceedings of the 40th International Conference on Software Engineering*, ICSE '18, page 705–715. ACM, May 2018. doi:[10.1145/3180155.3180193](https://doi.org/10.1145/3180155.3180193).
- [28] Stephen H. Edwards, Jason Snyder, Manuel A. Pérez-Quinones, Anthony Allevato, Dongkwan Kim, and Betsy Tretola. Comparing effective and ineffective behaviors of student programmers. In *Proceedings of the fifth international workshop on Computing education research workshop*, ICER '09, page 3–14. ACM, August 2009. doi:[10.1145/1584322.1584325](https://doi.org/10.1145/1584322.1584325).
- [29] Ayaan M. Kazerouni, Stephen H. Edwards, and Clifford A. Shaffer. Quantifying Incremental Development Practices and Their Relationship to Procrastination. In *Proceedings of the 2017 ACM Conference on International Computing Education Research*, ICER '17, page 191–199. ACM, August 2017. doi:[10.1145/3105726.3106180](https://doi.org/10.1145/3105726.3106180).
- [30] Miikka Kuuttila, Mika Mäntylä, Umar Farooq, and Maëlick Claes. Time pressure in software engineering: A systematic review. *Information and Software Technology*, 121:106257, May 2020. doi:[10.1016/j.infsof.2020.106257](https://doi.org/10.1016/j.infsof.2020.106257).
- [31] Zeinabsadat Saghi, Thomas Zimmermann, and Souti Chattopadhyay. Code Today, Deadline Tomorrow: Procrastination Among Software Developers. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, page 1204–1216. IEEE, April 2025. doi:[10.1109/icse55347.2025.00198](https://doi.org/10.1109/icse55347.2025.00198).
- [32] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590, 2023. URL <https://arxiv.org/abs/2302.06590>.
- [33] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. Measuring GitHub Copilot’s Impact on Productivity. *Communications of the ACM*, 67(3):54–63, February 2024. doi:[10.1145/3633453](https://doi.org/10.1145/3633453).
- [34] Shraddha Barke, Michael B. James, and Nadia Polikarpova. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):85–111, April 2023. doi:[10.1145/3586030](https://doi.org/10.1145/3586030).
- [35] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, page 1–13. ACM, February 2024. doi:[10.1145/3597503.3608128](https://doi.org/10.1145/3597503.3608128).
- [36] Muhammad Waseem, Aakash Ahmad, Kai-Kristian Kemell, Jussi Rasku, Sami Lahti, Kalle Mäkelä, and Pekka Abrahamsson. Vibe Coding in Practice: Flow, Technical Debt, and Guidelines for Sustainable Use. arXiv preprint arXiv:2512.11922, 2025. URL <https://arxiv.org/abs/2512.11922>.
- [37] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*, ITiCSE 2023, page 108–159. ACM, December 2023. doi:[10.1145/3623762.3633499](https://doi.org/10.1145/3623762.3633499).

- [38] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. Computing Education in the Era of Generative AI. *Communications of the ACM*, 67(2):56–67, January 2024. doi:[10.1145/3624720](https://doi.org/10.1145/3624720).
- [39] Michael Meli, Matthew R. McNiece, and Bradley Reaves. How Bad Can It Get? Characterizing Secret Leakage in Public GitHub Repositories. In *Proceedings 2019 Network and Distributed System Security Symposium*, NDSS 2019. Internet Society, 2019. doi:[10.14722/ndss.2019.23418](https://doi.org/10.14722/ndss.2019.23418).
- [40] Vibha Singhal Sinha, Diptikalyan Saha, Pankaj Dhoolia, Rohan Padhye, and Senthil Mani. Detecting and Mitigating Secret-Key Leaks in Source Code Repositories. In *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*, page 396–400. IEEE, May 2015. doi:[10.1109/msr.2015.48](https://doi.org/10.1109/msr.2015.48).
- [41] Setu Kumar Basak, Lorenzo Neil, Bradley Reaves, and Laurie Williams. SecretBench: A Dataset of Software Secrets. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, page 347–351. IEEE, May 2023. doi:[10.1109/msr59073.2023.00053](https://doi.org/10.1109/msr59073.2023.00053).
- [42] Setu Kumar Basak, Jamison Cox, Bradley Reaves, and Laurie Williams. A Comparative Study of Software Secrets Reporting by Secret Detection Tools. In *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, page 1–12. IEEE, October 2023. doi:[10.1109/esem56168.2023.10304853](https://doi.org/10.1109/esem56168.2023.10304853).
- [43] Alexander Krause, Jan H. Klemmer, Nicolas Huaman, Dominik Wermke, Yasemin Acar, and Sascha Fahl. Pushed by Accident: A Mixed-Methods Study on Strategies of Handling Secret Information in Source Code Repositories. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 2527–2544, Anaheim, CA, August 2023. USENIX Association. ISBN 978-1-939133-37-3. URL <https://www.usenix.org/conference/usenixsecurity23/presentation/krause>. Preprint: doi:[10.48550/arXiv.2211.06213](https://doi.org/10.48550/arXiv.2211.06213).
- [44] Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R. Lyu. Your Code Secret Belongs to Me: Neural Code Completion Tools Can Memorize Hard-Coded Credentials. *Proceedings of the ACM on Software Engineering*, 1(FSE):2515–2537, July 2024. doi:[10.1145/3660818](https://doi.org/10.1145/3660818).
- [45] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. Do Users Write More Insecure Code with AI Assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS ’23*, page 2785–2799. ACM, November 2023. doi:[10.1145/3576915.3623157](https://doi.org/10.1145/3576915.3623157).
- [46] Pinran Gao, Lingxiang Wang, Yi Liu, Kunpeng Liu, Fan Yang, and Ying Zhang. Mind your key: An Empirical Study of LLM API Credential Leakage in iOS Apps. arXiv preprint arXiv:2606.12212, 2026. URL <https://arxiv.org/abs/2606.12212>.
- [47] Songwen Zhao, Danqing Wang, Kexun Zhang, Jiaxuan Luo, Zhuo Li, and Lei Li. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks. arXiv preprint arXiv:2512.03262, 2025. URL <https://arxiv.org/abs/2512.03262>.
- [48] Junquan Deng, Zhiyu Fan, and Ruijie Meng. Understanding the (In)Security of Vibe-Coded Applications. arXiv preprint arXiv:2606.23130, 2026. URL <https://arxiv.org/abs/2606.23130>.
- [49] Qumash.kz. Five hours in the queue: HackAlem AI participants in Astana complained about the organisation, September 2026. URL <https://qumash.kz/news/pyat-chasov-v-ocheredi-uchastniki-hackalem-ai-v-astane-pozhalovalis-na-organizatsiyu/>. In Russian. Accessed 24 September 2026.
- [50] Ulys Media. “Survival simulator”: participants criticised HackAlem AI in Astana, September 2026. URL <https://ulysmedia.kz/news/81431-simuliator-vyzhivaniia-uchastniki-raskritikovali-hackalem-ai-v-astane/>. In Russian. Accessed 24 September 2026.
- [51] The Astana Times. Astana stages massive AI hackathon in Guinness World Records bid, September 2026. URL <https://astanatimes.com/2026/09/astana-stages-massive-ai-hackathon-in-guinness-world-records-bid/>. Accessed 24 September 2026.
- [52] Kazinform. What is built in five hours: how the largest AI hackathon in Astana is run, September 2026. URL <https://www.inform.kz/ru/chto-sozdayut-za-pyat-chasov-kak-prohodit-krupneyshiy-ai-hakaton-v-astane-72e380f9>. In Russian. Accessed 24 September 2026.
- [53] Gede Artha Azriadi Prana, Christoph Treude, Ferdian Thung, Thushari Atapattu, and David Lo. Categorizing the Content of GitHub README Files. *Empirical Software Engineering*, 24(3):1296–1327, October 2018. doi:[10.1007/s10664-018-9660-3](https://doi.org/10.1007/s10664-018-9660-3).
- [54] Gitleaks contributors. Gitleaks: a tool for detecting secrets in git repositories, version 8.30.1, 2026. URL <https://github.com/gitleaks/gitleaks>. Release v8.30.1, 21 March 2026. Accessed 24 September 2026.
- [55] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, November 1987. doi:[10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7).

- [56] Anthropic. Claude Code settings reference: git and attribution, 2026. URL <https://code.claude.com/docs/en/settings-reference>. States that commits get a git trailer such as Co-Authored-By by default. Accessed 27 September 2026.
- [57] Mukhammed Togmanov, Nurdaulet Mukhituly, Diana Turmakhan, Jonibek Mansurov, Maiya Goloburda, Akhmed Sakip, Zhuohan Xie, Yuxia Wang, Bekassyl Syzdykov, Nurkhan Laiyk, Alham Fikri Aji, Ekaterina Kochmar, Preslav Nakov, and Fajri Koto. KazMMLU: Evaluating Language Models on Kazakh, Russian, and Regional Knowledge of Kazakhstan. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 14403–14416. Association for Computational Linguistics, 2025. doi:[10.18653/v1/2025.acl-long.701](https://doi.org/10.18653/v1/2025.acl-long.701).
- [58] Paul Ralph, Nauman bin Ali, Sebastian Baltes, Domenico Bianculli, Jessica Diaz, Yvonne Dittrich, Neil Ernst, Michael Felderer, Robert Feldt, Antonio Filieri, Breno Bernard Nicolau de França, Carlo Alberto Furia, Greg Gay, Nicolas Gold, Daniel Graziotin, Pinjia He, Rashina Hoda, Natalia Juristo, Barbara Kitchenham, Valentina Lenarduzzi, Jorge Martínez, Jorge Melegati, Daniel Mendez, Tim Menzies, Jefferson Mollari, Dietmar Pfahl, Romain Robbes, Daniel Russo, Nyyti Saarimäki, Federica Sarro, Davide Taibi, Janet Siegmund, Diomidis Spinellis, Mirosław Staron, Klaas Stol, Margaret-Anne Storey, Davide Taibi, Damian Tamburri, Marco Torchiano, Christoph Treude, Burak Turhan, Xiaofeng Wang, and Sira Vegas. Empirical Standards for Software Engineering Research. arXiv preprint arXiv:2010.03525, 2020. URL <https://arxiv.org/abs/2010.03525>.
- [59] Olmo R. Van den Akker, Sara Weston, Lorne Campbell, Bill Chopik, Rodica Damian, Pamela Davis-Kean, Andrew Hall, Jessica Kosie, Elliott Kruse, Jerome Olsen, Stuart Ritchie, KD Valentine, Anna Van ’t Veer, and Marjan Bakker. Preregistration of secondary data analysis: A template and tutorial. *Meta-Psychology*, 5, November 2021. doi:[10.15626/mp.2020.2625](https://doi.org/10.15626/mp.2020.2625).